

Compte rendu :Medical Image Anomaly Detection

Contexte et objectif

Ce projet vise à détecter des anomalies (zones suspectes) dans des images médicales (IRM cérébrales) en comparant trois approches de traitement d'image croissantes en complexité : **séquentiel (CPU simple)**, **parallèle CPU (multithreading)**, et **accéléré GPU (CuPy)**. L'objectif affiché est double : démontrer une pipeline de détection de contours suspects, et comparer les performances des trois méthodes de calcul.

Le notebook est court (10 cellules) mais couvre l'ensemble de la chaîne : acquisition de données réelles, pipeline de traitement d'image classique (OpenCV), parallélisation CPU, puis portage GPU.

1. Préparation des données

Le notebook télécharge un vrai jeu de données médical via kagglehub : le dataset "**Brain MRI Images for Brain Tumor Detection**" (Navoneel, Kaggle), contenant des IRM cérébrales réparties en dossiers yes/no (présence ou absence de tumeur).

Deux points bien pensés dans cette partie :

- **Robustesse au téléchargement** : le code encapsule l'appel `kagglehub.dataset_download()` dans un `try/except`, avec `dataset_path = None` en cas d'échec — évite un plantage brutal si l'environnement n'a pas accès à Kaggle.
- **Génération d'images synthétiques de secours** (`generate_synthetic_image`) : une ellipse (simulant le cerveau) avec du bruit gaussien et un petit cercle plus clair (simulant une anomalie/tumeur), utilisée comme *fallback* si le dataset réel n'est pas disponible ou insuffisant. La fonction `load_mri_batch()` complète intelligemment un batch avec des images synthétiques si le nombre d'images réelles chargées est inférieur au nombre demandé.

D'après la sortie, le téléchargement a réussi (chemin local C:\Users\med31\.cache\kagglehub\...), donc l'exécution montrée utilise bien de vraies IRM et non les images synthétiques — bon signe pour la validité des résultats.

2. Pipeline séquentiel

python

```
def detect_anomalies_sequential(img):
```

```
    denoised = cv2.GaussianBlur(img, (5, 5), 0)
```

```
    _, thresh = cv2.threshold(denoised, 150, 255, cv2.THRESH_BINARY)
```

```
    contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

```
    return contours, denoised
```

La pipeline est une chaîne de traitement d'image classique et cohérente pour de la détection d'anomalies simple :

1. **Débruitage** par flou gaussien (réduit le bruit avant seuillage).
2. **Seuillage binaire** à 150 : tout pixel plus clair que 150 devient une zone "suspecte" (blanc), le reste devient noir — hypothèse implicite que les anomalies apparaissent comme des zones plus claires que le tissu environnant, ce qui est cohérent avec l'apparence typique de certaines tumeurs en IRM.
3. **Détection de contours externes** via `cv2.findContours`, qui délimite les zones suspectes identifiées.

Résultat mesuré : **0,0010 s** pour une image unique — cohérent avec la légèreté de la pipeline (pas de traitement lourd).

3. Parallélisme CPU

C'est la partie la plus intéressante du notebook d'un point de vue technique, car elle documente un **choix d'architecture corrigé et justifié** :

"Switched from ProcessPoolExecutor to ThreadPoolExecutor. ProcessPoolExecutor hangs in Jupyter because it needs to pickle detect_anomalies_sequential, which is defined in the notebook's __main__ namespace and can't be re-imported by worker processes."

C'est un point technique réel et bien identifié : ProcessPoolExecutor a besoin de sérialiser (pickle) la fonction et de la réimporter dans chaque processus worker, ce qui échoue silencieusement (ou bloque) si la fonction est définie dans une cellule Jupyter plutôt que dans un vrai module .py importable. Le passage à ThreadPoolExecutor est justifié par un argument technique valide : OpenCV **libère le GIL** pendant ses appels C++ internes, donc le multithreading Python peut réellement paralléliser les appels cv2.*, contrairement à du code Python pur qui resterait bridé par le GIL.

Résultats du benchmark :

Méthode	Temps
Séquentiel	0,0074 s
Parallèle (4 threads)	0,0040 s

Speedup $\approx 1,85\times$ — cohérent avec un gain de parallélisme réel mais partiel (attendu, car l'overhead de création du pool de threads pèse relativement lourd sur un batch aussi petit et une tâche aussi rapide).

4. Accélération GPU

python

```
def detect_anomalies_gpu(img):
```

```
    gpu_img = cp.asarray(img).astype(cp.float32)
```

```
    denoised = gaussian_filter(gpu_img, sigma=1)
```

```
    thresh = (denoised > 150) * 255
```

```
    thresh_cpu = cp.asnumpy(thresh).astype(np.uint8)
```

```
contours, _ = cv2.findContours(thresh_cpu, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

return contours, cp.asnumpy(denoised)
```

Le flou gaussien et le seuillage sont recalculés sur GPU via `cupyx.scipy.ndimage.gaussian_filter` (équivalent CUDA du filtre gaussien), tandis que `cv2.findContours` — qui n'a pas d'équivalent CuPy — est exécuté sur CPU après rapatriement des données (`cp.asnumpy`). C'est une architecture **hybride CPU/GPU** cohérente : seules les opérations matricielles lourdes (convolution gaussienne) sont déportées sur GPU, l'étape de détection de contours restant sur CPU car elle est intrinsèquement séquentielle/algorithmique.

Résultat : **0,0031 s** pour une image unique.

Synthèse critique des résultats

En comparant les trois temps sur une image unique : séquentiel CPU (0,0010 s) est en réalité **plus rapide** que le GPU (0,0031 s). Ce résultat, à première vue contre-intuitif, s'explique bien : pour une image 512×512, la charge de calcul est trop faible pour amortir le coût fixe de transfert CPU→GPU et d'initialisation CUDA. Le notebook ne relève pas explicitement cette conclusion — ce serait un ajout pertinent à faire dans les conclusions.

Points méthodologiques à améliorer pour un vrai benchmark scientifique :

1. **Pas de warm-up** avant les mesures GPU : la première exécution CuPy inclut typiquement l'initialisation du contexte CUDA, ce qui fausse fortement la mesure (contrairement au TP "Benchmark CPU vs GPU" précédent qui gérait bien ce point).
2. **Une seule mesure, pas de répétition/médiane** : les temps mesurés (`time.time()`) sur une seule exécution sont sensibles au bruit système ; une médiane sur plusieurs répétitions serait plus fiable.
3. **Pas de test à plusieurs tailles de batch** : le GPU n'est testé que sur 1 image. Il serait plus informatif de comparer CPU séquentiel, CPU parallèle et GPU sur des batches croissants (10, 100, 1000 images) pour identifier un vrai point de bascule, comme fait dans le TP CPU vs GPU précédent.

4. **Pas de synchronisation explicite** (`cp.cuda.Stream.synchronize()`) avant l'arrêt du chronomètre GPU — les appels CUDA étant asynchrones, le temps mesuré pourrait sous-estimer le temps réel de calcul si le rapatriement (`cp.asnumpy`) ne forçait pas déjà une synchronisation implicite (ce qui est heureusement le cas ici, car `asnumpy` bloque jusqu'à la fin du calcul).

Points forts

- Utilisation d'un **vrai dataset médical** (Kaggle) avec fallback synthétique robuste.
- Pipeline de traitement d'image cohérente et adaptée au cas d'usage (flou → seuil → contours).
- Bon diagnostic technique et correction documentée du problème `ProcessPoolExecutor` vs `ThreadPoolExecutor` — dénote une vraie compréhension du GIL et du multiprocessing en Python.
- Architecture hybride CPU/GPU pertinente (déporter seulement ce qui bénéficie du GPU).